arXiv:2401.08541v1 [cs.CV] 16 Jan 2024

Scalable Pre-training of Large Autoregressive Image Models

Alaaeldin El-Nouby Michal Klein Shuangfei Zhai Miguel Angel Bautista
Alexander Toshev Vaishaal Shankar Joshua M Susskind Armand Joulin*

Apple

https://github.com/apple/ml-aim

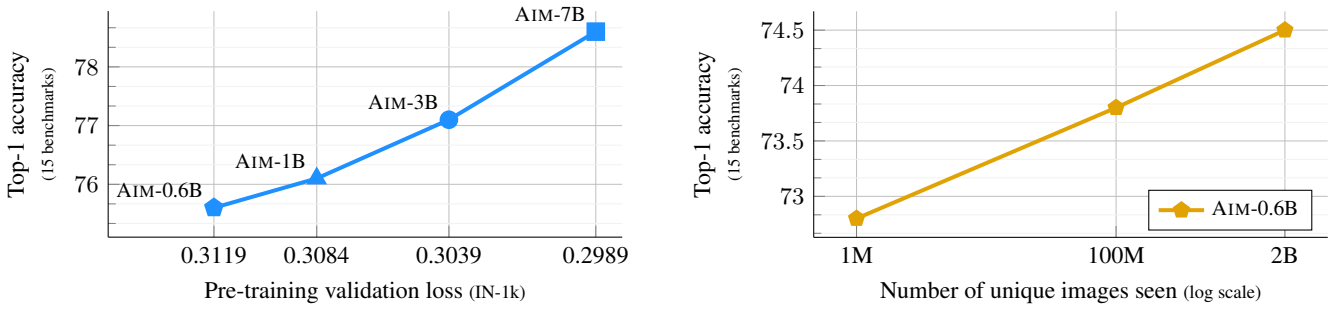


Figure 1. AIM scaling behavior (Left) As we scale the capacity of AIM, we observe improved performance for the pre-training objective which directly correlates with stronger downstream performance. (Right) AIM exhibits stronger downstream performance when trained using larger sets of uncurated web data [32, 33]. The downstream performance is the average attentive probe top-1 accuracy over a diverse set of **15 image recognition benchmarks**. All models are trained for the same number of updates.

Abstract

This paper introduces AIM*, a collection of vision models pre-trained with an autoregressive objective. These models are inspired by their textual counterparts, i.e., Large Language Models (LLMs), and exhibit similar scaling properties. Specifically, we highlight two key findings: (1) the performance of the visual features scale with both the model capacity and the quantity of data, (2) the value of the objective function correlates with the performance of the model on downstream tasks. We illustrate the practical implication of these findings by pre-training a 7 billion parameter* AIM *on 2 billion images that achieves 84.0% on ImageNet-1k with a frozen trunk. Interestingly, even at this scale, we observe no sign of saturation in performance, suggesting that* AIM *potentially represents a new frontier for training large-scale vision models. The pre-training of* AIM *is similar to the pre-training of LLMs, and does not require any image-specific strategy to stabilize the training at scale.*

1. Introduction

Pre-training task agnostic models has become the standard in Natural Language Processing with the recent revolution of large language models (LLMs) [13, 64, 75]. These models can solve complex reasoning tasks from a few examples [13], follow instructions [59], and now serve as the engine of widely used AI assistants such as ChatGPT. A key factor contributing to their success is the ability to consistently improve as the capacity (*i.e.*, number of parameters) or the amount of pre-training data [64] increases.

The scaling behavior of these models is remarkable for two key reasons. First, even though these models are trained with a simple objective – predicting the next word in a sentence given its past – they are able to learn intricate patterns over long contexts. Second, the scalability of this autoregressive objective is mostly observed when used in conjunction with certain architectures, and in particular Transformers [79], highlighting the potential synergy between the autoregressive pre-training and this architecture.

These observations naturally raise the follow-up question of whether the success of scaling Transformers with an autoregressive objective is exclusive to text. This is particularly significant considering that none of the aforemen-

*Work done while at Apple. Now at Google DeepMind.

tioned elements are inherently specific to language modeling. Autoregressive objectives take their roots in the data compression literature [69], and similar approaches have been investigated in audio [57] and images [18, 76]. The Transformer architecture has also been successfully used in other domains, in particular, computer vision with the success of the Vision Transformers (ViT) [29]. Therefore, as a first step towards generalizing the findings of LLMs, we explore if training ViT models with an autoregressive objective leads to competitive performance, in terms of learning representations, with the same scaling ability as LLMs.

In this paper, we introduce Autoregressive Image Models (**AIM**), an autoregressive approach for large-scale pre-training for visual features. We revisit prior work in autoregressive representation learning such as iGPT [18] using a modern toolset that includes vision transformers, collections of large-scale web data [32, 33] and recent advances in LLM pre-training [43, 75]. Additionally, we introduce two architectural modifications to adapt autoregressive pre-training to visual features. First, instead of restricting the self-attention to be fully causal as is typically the case for LLMs, we adopt a prefix attention, as in T5 [66]. This choice enables moving to a fully bidirectional attention during downstream tasks. Second, we use a heavily parameterized token-level prediction head, inspired by the heads used in contrastive learning [19]. We observe that this modification significantly improves the quality of the subsequent features with little overhead during training. Overall, the training of AIM is similar to the training of recent LLMs and does not rely on any stability-inducing techniques [24, 45, 74] that supervised [24, 74] or self-supervised [5, 58] methods need.

We provide a study of a series of models, ranging from 600M to 7B parameters pre-trained using 2B uncurated images with permissive licenses. Our AIM models exhibit strong scaling behavior w.r.t. the model size as shown in Figure 1 where higher capacity models achieve better downstream performance, measured as the average accuracy over 15 image recognition benchmarks. More importantly, there is a correlation between the value of our objective function on a validation set and the quality of the subsequent frozen features. This observation confirms that the autoregressive objective is adequate for the training of visual features. Furthermore, we observe consistent improvement in downstream performance as we train on more images, with no sign of saturation. Overall, these observations are aligned with the previous studies on scaling large language models.

2. Related Work

Autoregressive models. While most of the literature on autoregressive models come from language modeling [9, 53, 64] or speech [56, 57], few works have explored the potential of this approach for images [18, 49, 61, 61, 68, 76]. Of particular interest, Van den Oord et al. [76] show that using an architecture adapted to images, *e.g.*, a convolution network, significantly improved over autoregressive models built with more generic architecture [77], *e.g.*, a recurrent network [31]. Parmar et al. [61] further improve the quality of these autoregressive models by adopting the transformer architecture [79]. More recently, Chen et al. [18] have shown that scaling with more compute leads to continuous improvements. Our work follows this line of research, and we benefit from training on significantly more data, and further improvement in architecture design [29], training [73, 75] and understanding of the scaling law [43]. Concurrent to our work, Bai et al. [3] demonstrate the effectiveness of large-scale autoregressive vision models for in-context pixel prediction tasks (*e.g.*, semantic segmentation, depth estimation).

Self-supervised pre-training. Pre-training vision models on datasets of images without supervision has been a fruitful area of research in recent years [10, 27, 28, 34, 54, 87, 88]. Different approaches have been employed, focusing on various proxy tasks for feature learning. For example, Noroozi and Favaro [55] learn to re-arrange the order of shuffled image patches. Some other works have relied on clustering [7, 14, 17, 83]. Another popular approach involves the use of a contrastive objective, that resembles predictive coding, where the objective is to identify each image [19, 40]. Most recent contrastive approaches include DINO [58], BYOL [38] or iBot [88]. In a similar vein, some works have proposed predictive approaches [2, 6] or a form of feature whitening [85]. Closer to our approach are works inspired by BERT [26] where patches are masked and predicted with an autoencoder in either their discrete [5] or pixel [41] form.

Other generative pre-training. Autoregressive modeling is a form of generative modeling, and few other generative approaches have been considered to learn visual features. The first category leverages some form of autoencoding where the pretext task corresponds to some denoising task. For instance, the noise can be salt-and-pepper [81] or masking [5, 62]. Another line of work leverages Generative Adversarial Networks (GANs) [35]. Most notably, BigGAN [12] trains a large GAN and re-uses the image discriminator to produce image features. More recently, DiffMAE [82] used diffusion models to learn image features.

Pre-training at scale. There are numerous works on scaling the pre-training of visual features with no supervision [15, 36, 37, 58, 70, 72]. The most salient work in this area is DINOv2 where they produce the best self-supervised features by scaling the iBot method [88] on a private dataset of 142M images and a 460M parameter model. The conclusion from this work is that a carefully tuned contrastive method scales reasonably well, but they do not exhibit the scaling law that we observe with language modeling. They

also rely on an intricate implementation of contrastive learning to avoid the pitfalls described by Chen et al. [20]. In parallel, Singh et al. [70] study the scaling of Masked Autoencoders (MAE) [39]. While the study focuses on a weakly-supervised setup, it does not showcase strong improvements to the self-supervised pre-training as the data is scaled to billions of images. In contrast, we observe a clear benefit of scale on the quality of our features, even at a scale of a few billions of parameters and billions of images.

3. Pre-training Dataset

We pre-train our models on the DFN dataset introduced by Fang et al. [32]. This dataset is composed of a larger collection of 12.8B image-text pairs [33] filtered from Common Crawl. The data has been pre-processed to remove NSFW content, blur faces, and reduce contamination by deduplicating against the evaluation sets. A data filtering network [32] ranks the samples in the 12.8B collection according to the alignment score between images and their corresponding caption. A subset of 2B images, called DFN-2B, has been extracted from the DataComp 12.8B dataset [33] by keeping the top 15% samples. Note that other than the privacy and safety filters, this process does not include any additional curation based on the image content. Since our pre-training does not require text, our method could be pre-trained using larger image collections that are not paired with captions or have low image-text alignment such as the rest of DataComp 12.8B.

Motivated by the common practice in LLM pre-training [75] of oversampling high-quality data sources such as Wikipedia and Books, during pre-training, we sample images from DFN-2B with a probability of $p = 0.8$ and sample images from ImageNet-1k with a probability of $p = 0.2$. We refer to such dataset as DFN-2B+.

4. Approach

4.1. Training Objective

Our training objective follows that of a standard autoregressive model applied on a sequence of image patches. More precisely, an image x is split into a grid of K non-overlapping patches $x_k, k \in [1, K]$, which collectively form a sequence of tokens. We assume that the sequence order is fixed across all images, and we use a raster (row-major) ordering by default unless otherwise specified. Given the above order, the probability of an image can be factorized as a product of patch conditional probabilities:

$$P(x) = \prod_{k=1}^{K} P(x_k \mid x_{<k}), \tag{1}$$

where $x_{<k}$ denotes the set of the first $k-1$ patches, and is the context used to predict the kth patch. As opposed to language modeling, our sequences have a fixed length of K

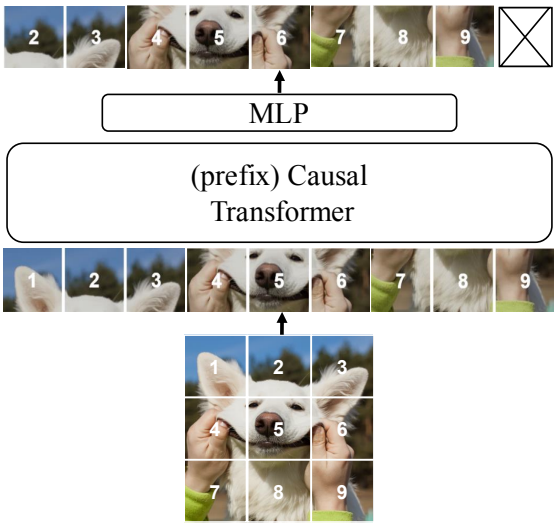


Figure 2. AIM pre-training overview.. Input images are split into non-overlapping patches and embedded linearly following Dosovitskiy et al. [29]. The patch features are fed to a transformer in which the self-attention operation is causally masked to prevent attending to preceding positions. Afterward, a heavily parameterized MLP processes each of the patch features independently and finally projects it to pixel space. The targets correspond to the input sequence shifted one position to the left, requiring the model to predict the next patch in raster order.

that fits in memory and hence we do not need to truncate the context length. The training loss over a set $\mathcal{X}$ of images is then defined as the negative log-likelihood (NLL):

$$\sum_{x \in \mathcal{X}} \sum_{k=1}^{K} -\log P(x_k \mid x_{<k}).$$

Minimizing this objective over an infinite amount of images, with no further assumptions, is theoretically equivalent to learning the true underlying image distribution.

Prediction loss Our training objective naturally gives rise to certain variants of losses, each corresponding to a choice of the distribution $P(x_k \mid x_{<k})$. By default, we adopt a normalized pixel-level regression loss similar to He et al. [41]. This loss corresponds to setting $P(x_k \mid x_{<k})$ as Gaussian distributions with a constant variance. Namely, given $\hat{x}_k(\theta)$ as the prediction of the kth patch from a network parameterized with θ, and x_k as its corresponding ground-truth value, our objective is to minimize the sum ℓ_2 squared distance between the prediction and the ground-truth:

$$\min_{\theta} \frac{1}{K} \sum_{k=1}^{K} \|\hat{x}_k(\theta) - x_k\|_2^2. \tag{2}$$

We also consider a cross-entropy loss with patches converted to discrete tokens using an offline tokenizer. Our ablation studies show that these designs work, although they do not produce as strong features as the pixel-wise loss.

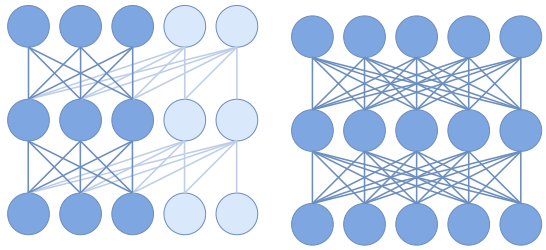


Figure 3. Prefix causal attention. During pre-training we uniformly sample a prefix length S. The attention for the first S patches are set to be bidirectional and loss is only computed for the remaining patches in the image. During adaptation to downstream tasks, this allows us to drop the attention causal mask, improving the downstream performance.

Model	#Params	Hidden size	Layers	LR	#Patches	Batch size
AIM-0.6B	0.6B	1536	24	$1e^{-3}$	0.5T	4096
AIM-1B	1.2B	2048	24	$1e^{-3}$	1.2T	4096
AIM-3B	2.7B	3072	24	$1e^{-3}$	1.2T	4096
AIM-7B	6.5B	4096	32	$1e^{-3}$	1.2T	4096

Table 1. Model specifications. We provide the embedding dimension, number of layers, and parameter count for all AIM variants. We also provide the learning rate and batch size during pre-training. For AIM with 1B parameters and higher, the pre-training process involves 1.2M iterations, which corresponds to 1.2 trillion patches, or 5B images, seen during pre-training.

4.2. Architecture

As the backbone, we adopt the Vision Transformer architecture (ViT) [28]. For scaling in the model capacity, we follow the common practice in language modeling and we prioritize expanding width rather than depth [64, 75]. In Table 1, we provide an overview of the design parameters of AIM, including its depth and width, as well as the amount of data and optimization scheme for each model capacity. The overall model is illustrated in Figure 2.

During pre-training, we apply causal masks to the self-attention layers to model the probability of a patch given the preceding patches. More precisely, given a self-attention layer, the embedding for the patch i is computed by:

$$y_i = \sum_{k=1}^{K} a_{ik} v_i, \tag{3}$$

where a_{ik} is the attention weight and v_k the value embedding. To enforce the desired constraints, we utilize a causal mask for the attention weights, where $a_{ik} = 0$ for $k > i$, and $\sum_{k=1}^{K} a_{ik} = 1$. This approach enables us to process the image with a single forward pass during training, without incurring additional computational overhead.

Prefix Transformer. The autoregressive objective in pre-training requires a causal mask in the self-attention operation. However, this differs from the standard usage of ViT models in downstream tasks, where bidirectional self-attention is employed. This discrepancy leads to a decrease in performance, irrespective of whether the causal mask is retained during downstream adaptation or not (as shown in the ablations presented in Table 3). To address this issue, we propose to consider the initial patches of the sequence, referred to as the prefix, as a context for predicting the remaining patches following the PrefixLM formulation of Raffel et al. [65]. The prefix patches are excluded from the autoregressive prediction and therefore are not constrained to be causal. More precisely, we select a prefix length of size $S \in [1, K-1]$, and remove the causal mask, *i.e.*, $a_{i,k} > 0$ for $k < S$. This modification helps the model to work in the absence of causal masking, allowing it to be removed during downstream adaptation. This approach improves the performance of the model in downstream tasks and eliminates the need for architectural changes to ViT. Figure 3 illustrates the difference between causal and prefix attention.

MLP prediction heads. It is a common practice to adopt certain prediction heads during pre-training, which are discarded when transferring to downstream tasks [16, 17, 19, 20, 38]. The purpose of these heads is to prevent the trunk features from becoming too specialized in the pre-training objective, thus enhancing their suitability for downstream transfer. We opt for a simple design where we use N blocks of MLP on top of the final transformer layer, processing each patch independently. We observed that this design strikes a good balance between performance and the additional costs incurred during pre-training.

Straightforward implementation. It is worth noting that AIM does not require particular optimization stability-inducing mechanisms such as LayerScale [74], stochastic depth [45], QK-Norm [24], or freezing the patch projector [20]. These mechanisms have been crucial for the success of other methods, either supervised or self-supervised. On the contrary, we observe that AIM scales using the same set of optimization hyperparameters across model sizes with no further tuning (see Table 1).

We add sinusoidal positional embeddings [79] to the input patches before the transformer and before the MLP head. We use a standard expansion ratio of 4 for all the MLP blocks in the trunk and the head. We drop the bias term for simplicity, and unlike the original ViT, we do not append a classification token to the input. By default, we use 12 blocks for the MLP head for all model capacities. The pixel targets are normalized per patch before the loss computation following He et al. [41]. We train our model using `bfloat16` precision. We use the AdamW [52] optimizer with linear warmup and a cosine decay schedule. We detail the hyperparameters used for pre-training and downstream adaptation in Appendix D.

Downstream adaptation. Pre-training large-scale models is a resource-intensive process, and even fine-tuning them

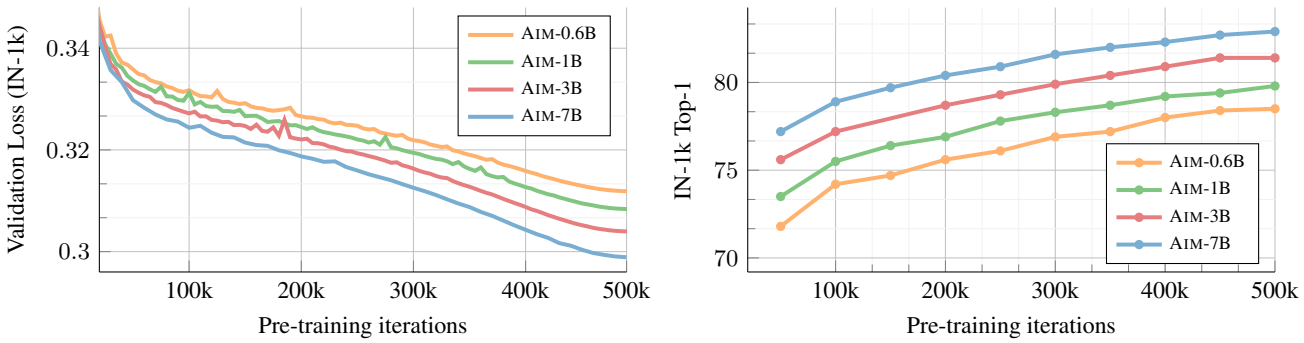


Figure 4. AIM pre-training across model sizes. We observe a clear improvement in the performance of the pre-training objective with increasing the capacity of AIM. Moreover, the downstream performance (IN-1k top-1) is monotonically improving for higher capacity models as well as with longer pre-training. We do not observe clear signs of plateauing during pre-training even after training for 500k iterations, indicating that AIM can benefit from even longer pre-training schedules. Note that the loss saturation at the very end of training is caused by the cosine decay schedule where the learning rate is effectively zero.

is demanding. Consequently, we focus on scenarios where all model weights are fixed for downstream tasks. In this context, we only train a classification head, which mitigates the risk of overfitting on small downstream datasets and significantly reduces the adaptation cost.

Unlike contrastive learning, our loss is computed independently for each patch. This means that our pre-training does not incorporate any notion of global image descriptors, and hence, we do not have any image level token. While some methods rely on global average pooling to build a global feature from the patch features, we find that our approach, along with other generative approaches like MAE, benefit more from an attention pooling operation [50] placed before the linear classifier. Other works [1, 74, 84] have adopted this attention pooling to improve performance with minimal increase in parameters or FLOPs.

Specifically, given a set of patch features $P = \{p_i \mid 1 \leq i \leq K\}$, we compute a global descriptor $\hat{p}$ through multi-head attention pooling over the patch features as:

$$\hat{p_h} = \sum_{i=1}^{K} \frac{\exp(q_h^T W_h^k p_i)}{\sum_{j=1}^{K} \exp(q_h^T W_h^k p_j)} W_h^v p_i, \tag{4}$$

where for each attention head $h = \{1, ..., H\}$, $W_h^k, W_h^v \in R^{d_h \times d}$ correspond to the key and value weight matrices, respectively; q_h is a learnable query vector. And we obtain the pooled feature as $\hat{p} = [p_1, ..., p_H], \hat{p} \in R^d$, which serves as the input to the linear classifier. By default, we set the number of heads $H = \frac{d}{d_h}$, which makes the total number of learnable parameters $2d^2 + d$, a negligible cost compared to the main model size. Including this attention pooling makes the entire operation not strictly linear, and, therefore we refer to it as "Attentive Probe". Nevertheless, the advantages of linear probing, e.g., low additional parameter count and a reduced risk of overfitting, are preserved with this probe.

5. Results

5.1. Impact of scaling

We measure the impact when scaling our approach in terms of parameters and training data. In particular, we investigate whether there is a correlation between the pre-training objective and the downstream performance across benchmarks. We also look at the effect of scaling on the value of the loss function. For all of these experiments, we report the value of our loss function on the validation set of IN-1k.

Loss and performance during training. In Figure 4, we measure for each model the value of the pre-training loss and the classification accuracy on the validations set, as a function of the number of training iterations. We observe that both probes improve accordingly during the entire training, showing that optimizing our objective directly results in better downstream performance.

Number of parameters. We observe that the loss value and the accuracy of the downstream task improve as we scale the capacity of our models. This observation is consistent with the trend observed in LLMs and can be directly attributed to the optimization of our objective function, which in turn leads to the learning of stronger representations.

Number of images. In Figure 5, we show the progression of the validation loss as we pre-train on either a small curated dataset of 1M images, *i.e.*, IN-1k, or a larger set of 2B images, *i.e.* DFN-2B+. It is not surprising that training on IN-1k leads rapidly to a low validation loss as measured on the same distribution. However, this loss deteriorates at the end of the training, indicating an overfitting to the training data. When training on the uncurated DFN-2B dataset, the model starts from a higher validation loss but the loss continues to decrease with no sign of overfitting. When the same dataset is augmented with a small amount of IN-1k data, as detailed in § 3, we observe further improvement in

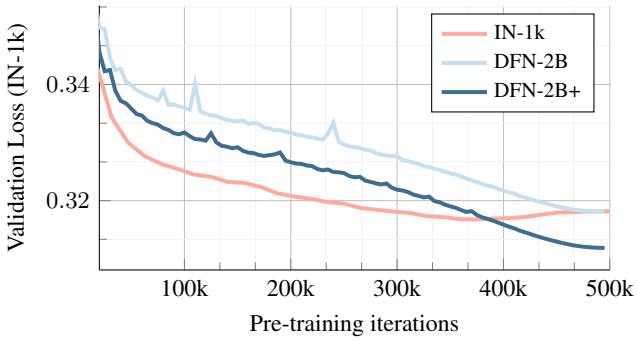


Figure 5. Dataset impact on pre-training performance. On the one hand, pre-training using IN-1k leads to overfitting, even for the AIM-0.6B model. On the other hand, pre-training using the uncurated DFN-2B dataset prevents overfitting but converges to a similar point due to the distributional shift. Pre-training on DFN-2B+, a data mixture that predominantly consists of DFN-2B with a small presence of IN-1k samples leads to the best performance.

the performance that eventually surpasses pre-training on IN-1k. We confirm that the resulting model also leads to a better downstream performance in Table 2.

pre-training dataset	IN-1k	DFN-2B	DFN-2B+
attentive	73.5	74.5	**75.6**

Table 2. Dataset impact of downstream performance (15 benchmarks). The behavior in Figure 5 is consistent with the downstream performance where we observe that using a data mixture of DFN-2B and IN-1k results in the best performance.

Compute-optimal pre-training. Since we do not observe signs of overfitting when we train using the DFN-2B+ dataset, we proceed to examine the impact of extending the length of our pre-training schedule. In Figure 6, we study the impact of increasing the length of the pre-training schedule from 500k to 1.2M iterations, *i.e.*, 2B to 5B images seen during pre-training. We observe that models pre-trained with a longer schedule achieve significantly lower validation loss. This suggests that one can improve the performance of AIM either by increasing the model capacity or by pre-training for longer schedules. Interestingly, we find that lower-capacity models trained for a longer schedule achieve comparable validation loss to higher-capacity models trained for a shorter schedule while using a similar amount of FLOPs. This finding is consistent with Hoffmann et al. [43] and implies that AIM could follow similar scaling laws. However, we defer further investigations in this aspect for future work.

5.2. Architecture and Design

In this section, we investigate the impact of some variations in our model and training objective. These ablations are conducted using an AIM-0.6B model, which has been pre-trained and evaluated on the IN-1k dataset. The results of these ablations are presented in Table 3.

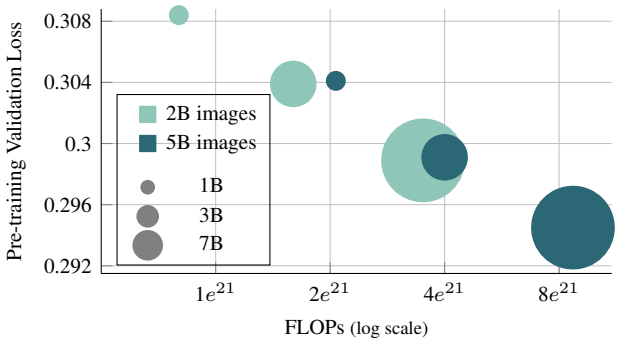


Figure 6. Scaling in FLOPs. That total number of FLOPs during training correlates with the final validation loss, suggesting compute driven scaling law similar to Hoffmann et al. [43].

Targets and objective (a). We explore various potential representations for the target patches. One approach is to utilize the raw pixel values, and training the model with mean squared error (MSE) regression loss. A second option, proposed by He et al. [41], involves using per-patch normalized pixel values instead of the raw signal with the same MSE loss. Finally, another option is to use a discretized representation of the patches, either using k-means or a discrete VAE [67, 78]. In this case, the model is trained using a cross-entropy objective similar to language modeling. Our experiments show that AIM performs best when using the MSE objective with normalized pixel values.

Autoregression pattern (b). Autoregressive pre-training typically follows a specific order of traversal to facilitate the prediction of the next token. In the case of language, the traversal pattern is clear, as text is read and written one word at a time in a sequential manner (*e.g.*, left to right for English). However, for images, determining the traversal pattern is less obvious. We explore various deterministic patterns, including raster, spiraling out, checkerboard, and randomly pre-sampled patterns. Detailed examples of each pattern are found in Appendix B. Even though our model performs reasonably well with each pattern, we observe that the raster pattern leads to significantly higher performance.

To gain deeper insights into this result, we examine the difficulty of predicting patches along sequences for each pattern. This can be done by measuring the loss value per patch as we progress along a sequence, as illustrated in Figure 7. Our observation is that patterns that present a more uniform distribution of difficulty across patches result in superior models, as compared to patterns where the prediction becomes progressively easier as the sequence unfolds. We attribute this to the difficulty of predicting patches throughout the sequence that forces the model to retain more information about the image. This leads to better patch features, and consequently, to better image representation as a whole.

Cropping scale (c). We explore the impact of the information content of each patch by adjusting the lower bound of the cropping scale. On the one hand, opting for a crop-

target	pixels	norm. pixel [41]	KMeans	dVAE [67]
linear	67.5	**70.0**	66.6	64.0
attentive	76.2	**78.2**	75.9	74.5

(a) Targets.

pattern	raster	spiral	checkerboard	random
linear	**69.5**	67.7	68.2	65.8
attentive	**77.4**	76.3	76.0	75.7

(b) Autoregression Pattern (causal).

crop scale	0.08	0.4	1.0
linear	68.4	**70.0**	49.6
attentive	77.7	**78.2**	63.5

(c) Crop Scale.

pre-training attn.	causal		prefix	
inference attn.	causal	bidirectional	causal	bidirectional
linear	69.5	30.9	68.4	**70.0**
attentive	77.4	52.3	76.9	**78.2**

(d) Attention Structure.

head	None	MLP	Transformer
linear	64.0	70.0	**70.5**
attentive	75.4	78.2	**78.5**

(e) Head Design.

architecture	deep	wide
linear	68.8	**70.0**
attentive	77.9	**78.2**

(f) Architecture.

Table 3. Ablations We investigate various design choices of AIM. We use an AIM-0.6B model that is pre-trained and evaluated using IN-1k. We report the linear and attentive probing results. The default settings for AIM used for the main results are highlighted in gray.

ping scale that is too small leads to an easier next-patch-prediction task as neighboring patches' similarity increases. On the other hand, using a large cropping scale can lead to severe overfitting unless the dataset size is sufficiently large. Since this study is conducted using IN-1k, we observe a clear drop in performance due to overfitting.

Causal *vs*. Prefix Attention (d). We measure the impact of incorporating prefix attention during pre-training, as opposed to using standard causal attention. We observe that pre-training with causal self-attention produces models that are effective in downstream transfer tasks only when the causal mask is preserved. These models experience a significant decline in performance when bidirectional attention is employed. However, pre-training with prefix attention leads to models that operate effectively in both causal and bidirectional modes. Notably, the best performance is achieved when combining prefix attention during pre-training with bidirectional attention during downstream adaptation.

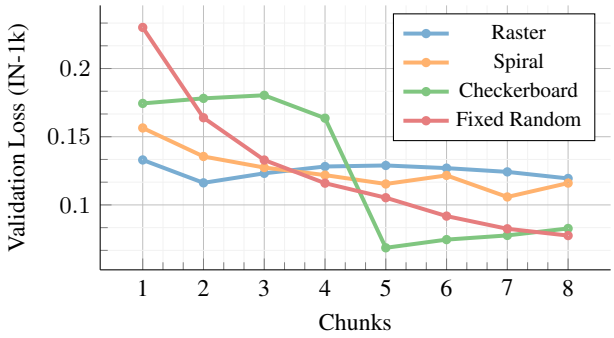


Figure 7. Autoregression patterns We explore a number of patterns for the autoregressive traversal of an image. The set of image patches is broken into equal-sized chunks and the validation loss is measured per chunk. We observe that the way the task difficulty is distributed across chunks varies strongly among patterns.

Head design (e). We consider different types of heads on top of the backbone to make predictions at the pixel level. Using no heads (*i.e. None*) performs reasonably well, but adding an MLP further improves the quality of the backbone. Interestingly, replacing the MLP with a full-fledged transformer of the same depth and width only yields a marginal performance improvement but at a significantly higher computational cost. Therefore, we opt to use an MLP head in our approach. We hypothesize that these heads specialize in capturing the low-level signals necessary for accurate pixel-level prediction. By incorporating a head, the trunk can learn higher-level features that are more suitable for downstream transfer. A similar design was employed for contrastive learning to prevent the backbone from specializing in predicting specific image transformations [19].

width	512	1024	2048
linear	69.4	69.6	**70.0**
attentive	77.7	78.1	**78.2**

(a) MLP width.

depth	6	8	12
linear	65.3	68.1	**70.0**
attentive	76.2	77.1	**78.2**

(b) MLP depth.

Table 4. MLP design. We vary the capacity of the MLP head by changing the number of MLP blocks (*i.e.* depth) or the embedding size (*i.e.* width). Downstream performance improves with more capacity in either width or depth, but depth has more impact.

	autoregressive	masked image modeling	
		ratio=50%	ratio=75%
attentive	**78.2**	70.3	77.8

Table 5. Autoregressive *vs*. Masking We evaluate the IN-1k performance of the autoregressive objective of AIM, in comparison to the masking objective [5, 26]. We keep all the other architectural and optimization components fixed. We observe that, under the same pre-training settings, the frozen-trunk performance of the autoregressive objective outperforms masking.

Deeper *vs*. Wider architecture (f). We present the design specifications of AIM in Table 1, outlining its width and depth. Unlike the original design of ViT [29], where the depth is scaled more rapidly than the width, we adopt a scaling strategy similar to that of Llama [75]. This allows us to scale our model more gracefully while maintaining a reasonable depth. We validate the effectiveness of a wider architecture in Table 3f. Our findings indicate that even for the relatively small-scale AIM-0.6B model, a wider architecture not only delivers strong performance but also im-

Model	Arch.	Data	IN-1k	iNAT-18	Cifar10	Cifar100	Food101	DTD	Pets	Cars	iWildCam	Camelyon17	PCAM	RxRX1	EuroSAT	fMoW	Infographic	Avg
DINO [17]	ViT-B/8	IN-1k	80.1	66.0	97.8	87.3	89.5	78.4	92.3	89.2	58.5	93.7	90.2	6.1	98.2	57.0	41.1	75.0
iBOT [88]	ViT-L/16	IN-21k	83.5	70.5	99.2	93.3	93.5	81.6	92.8	90.8	61.8	94.5	90.0	5.9	98.0	60.3	47.7	77.6
DINOv2 [58]	ViT-g/14_{516}	LVD	86.4	84.5	99.6	95.2	96.3	86.3	96.4	95.6	68.2	96.5	90.7	8.0	98.6	66.7	58.8	81.9
BEiT [5]	ViT-L/14	IN-21k	62.2	44.4	94.4	78.7	79.0	64.0	80.9	69.5	52.0	92.8	88.2	4.2	97.5	47.7	25.9	65.4
MAE [41, 70]	ViT-H/14	IN-1k	80.9	64.6	97.1	85.8	90.2	78.1	95.0	93.7	58.1	94.2	89.8	5.4	98.1	56.9	42.2	75.3
	ViT-2B/14	IG-3B	82.2	70.8	97.5	87.3	93.4	81.2	95.1	94.9	57.8	94.4	90.3	7.3	98.2	60.1	50.2	77.4
AIM-0.6B	ViT-H/14	DFN-2B+	78.5	64.0	97.2	86.8	90.1	80.1	93.0	93.0	57.9	94.3	90.0	7.8	98.4	58.3	45.2	75.6
AIM-1B	ViT-1B/14		80.6	67.2	98.2	88.3	91.6	81.8	93.4	93.9	58.6	94.5	90.0	9.0	98.6	59.8	47.5	76.9
AIM-3B	ViT-3B/14		82.2	69.7	98.4	89.9	92.7	81.9	94.1	93.8	58.8	94.3	90.4	9.7	98.5	60.9	48.9	77.6
AIM-7B	ViT-7B/14		82.4	70.9	98.6	90.0	93.1	82.3	93.8	92.1	59.5	93.6	90.7	10.1	98.6	61.7	49.6	77.8
AIM-7B†	ViT-7B/14	DFN-2B+	84.0	75.5	98.9	91.8	94.1	85.6	95.4	95.0	61.4	94.2	90.5	8.4	98.5	63.5	57.7	79.6

Table 6. Downstream evaluation with a frozen trunk. We assess the quality of AIM features by evaluating against a diverse set of 15 image recognition benchmarks. AIM and the baseline methods are evaluated using attentive probing with a frozen trunk. AIM models exhibit a strong performance across all benchmarks, especially the AIM-7B. AIM outperforms all other methods, using joint-embedding or generative approaches, except for DINOv2 which utilizes higher-resolution images, that typically results in a 1-1.5% improvement on ImageNet for instance. †: Extracting features from the 20th layer instead of the last (32nd), see Table 7 for more details.

proves training stability. This observation supports the notion that some of the insights gained from training LLMs can be similarly applied to other domains.

Attentive *vs*. Linear probe. For all ablations we report the linear and attentive probing results. We observe that, consistently across all experiments, attentive pooling provides a significant boost to performance as it allows for a more nuanced aggregation of local features circumventing one of the main weaknesses of generative pre-training: the absence of an image-level global descriptor.

Structure of the MLP. The MLP plays an important role as ablated in Table 3e. In Table 4, we further investigate the capacity of the MLP head and how it impacts downstream performance. We vary the capacity of the head by either changing the number of MLP blocks or their width. By default, we use a head of 12 blocks and an embedding dimension of 2048. First, we observe that increasing the capacity of the MLP either through depth or width leads to consistent improvement in the downstream performance. Second, we find that increasing the number of MLP blocks, with a fixed width, leads to a larger improvement compared to increasing the width for a fixed depth. Interestingly, we could not find a point where increasing the MLP capacity failed to yield further improvements. We did not explore higher capacities beyond those reported in Table 4 as it would lead to models with disproportionate head and trunk capacity.

5.3. Pre-training objective

Autoregressive *vs*. Masking We conduct a comparison between our architecture trained with an autoregressive objective and the masking objective popularized by BERT [26] for language, and by BEiT and MAE for vision. It is important to note that we applied the masking objective in the same setting as AIM, thereby isolating the impact on the performance of the pre-training objective from other design choices that differ between AIM and other approaches. In the masking baseline, we randomly sample masks and replace the masked patches with learnable mask tokens.

In Table 5, we show that AIM performs better with an autoregressive objective than a masking objective. This is consistent with the results reported by Chen et al. [18], providing further evidence that our improvements stem from the utilization of an autoregressive objective.

5.4. Comparison with other methods

In Table 6, we compare the attentive probing performance of AIM to other state-of-the-art methods across a set of 15 diverse benchmarks that are detailed in Appendix A.

Generative methods. AIM provides a strong performance compared to its generative counterparts. AIM outperforms BEiT [5] by a large margin. Additionally, AIM-0.6B provides a better performance, averaged across all benchmarks, compared to MAE-H [41] which has an equivalent capacity. Moreover, we compare against the MAE-2B [70] model which has been pre-trained on IG-3B, a private dataset of 3 billion images from Instagram. We find that both AIM-3B and AIM-7B outperform MAE-2B, with AIM-7B exhibiting a particularly large improvement. It is worth noting that, similar to AIM, two other generative approaches, BEiT and MAE, benefit from attentive probing, thereby narrowing the gap between generative and joint embedding methods.

Joint embedding methods. AIM provides a competitive performance with joint embedding methods such as DINO [17], iBOT [88], and DINOv2 [58]. In terms of

average accuracy across all benchmarks, AIM outperforms DINO and iBOT. However, it falls behind DINOv2 which achieves its results by evaluating with higher-resolution inputs. Note that AIM attains such competitive performance using higher capacity trunks. Nevertheless, AIM's pre-training is significantly simpler and can be trivially scaled in terms of parameters and data, yielding consistent improvements. On the contrary, state-of-the-art joint embedding methods like DINOv2 heavily rely on a number of tricks, such as multi-crop augmentation, KoLeo regularization, LayerScale, Stochastic Depth, schedules for teacher momentum and weight decay, and high-resolution fine-tuning in order to achieve strong performance.

Extracting stronger features. We observe that higher-quality features can be extracted from shallower layers compared to the last layer's features. This is likely due to the generative nature of the pre-training objective that is inherently different than the discriminative downstream tasks and therefore, the features with the highest semantic content do not necessarily concentrate around the last layer. In Table 7, we report the IN-1k top-1 accuracy for features extracted from the last layer compared to the layer with the highest performance. A more detailed analysis of this phenomenon is provided in Appendix D.

	AIM-0.6B	AIM-1B	AIM-3B	AIM-7B
last layer	78.5	80.6	82.2	82.4
best layer	**79.4**	**82.3**	**83.3**	**84.0**

Table 7. Feature extraction. The highest quality features after AIM pre-training typically reside in shallower layers than the last. Extracting features from earlier layers leads to a non-negligible boost to the recognition performance on IN-1k.

5.5. Low-Rank Adaptation

In addition to frozen-trunk evaluation, we examine Low-Rank Adaptation (LoRA) [44], a popular and efficient finetuning method. We report the results of LoRA fintuning of AIM in Table 8. We observe that LoRA is compatible with AIM, leading to a large boost in performance compared to frozen-trunk evaluation. For example, AIM-7B improves by 3.9% (compared to the last layer's performance) while finetuning only 0.1% percent of the trunk parameters.

	AIM-0.6B	AIM-1B	AIM-3B	AIM-7B
attentive	78.5	80.6	82.2	82.4
LoRA (rank=8)	81.0	83.6	85.5	86.3

Table 8. Low-rank adaptation (IN-1k). AIM is compatible with LoRA showing large gains compared to frozen-trunk evaluations.

6. Discussion

In this paper, we presented a simple and scalable method for pre-training vision models at scale without supervision. We employed a generative autoregressive objective during pre-training and proposed several technical contributions to better adapt it for downstream transfer. Consequently, we observed a number of desirable properties for our Autoregressive Image Models. First, the capacity of our models can be effortlessly scaled to 7 billion parameters using a vanilla transformer implementation, without resorting to stability-inducing techniques or extensive adjustments of hyperparameters for each model scale. Second, AIM's performance on the pre-training task has a strong correlation with downstream performance. Third, AIM achieves strong performance across 15 recognition benchmarks, outperforming prior state-of-the-art methods like MAE and significantly narrowing the gap between generative and joint embedding pre-training approaches. Finally, we did not observe any clear signs of saturation as we scale either in terms of parameters or data, suggesting that there is a potential for further performance improvements with larger models trained for even longer schedules. We hope that AIM serves as a seed for future research in scalable vision models that *effectively leverage uncurated datasets* without any bias towards object-centric images or strong dependence on captions.

Limitations. AIM excels in its seamless scalability and its effective utilization of large volumes of uncurated image data. However, alternative methods can offer different trade-offs. MAE [41] provides high sample efficiency and can learn good representations using a small amount of pre-training data, reducing the risk of overfitting [30] in contrast to our approach. Contrastive methods [17, 58, 88] currently result in stronger representations for a given model size compared to generative approaches such as MAE and AIM, but pose significant challenges in terms of scalability and loss tractability due to the complexity of their objective.

Acknowledgements

The authors would like to thank Brandon McKinzie, Samira Abnar, Preetum Nakkiran, and Jiatao Gu for valuable feedback throughout the project. We thank Edouard Grave and Hervé Jegou for their inspiring discussions during the earlier stages of the project. We thank Marco Cuturi, James Thornton, Pierre Ablin, and Eugene Ndiaye for their support and for many fruitful discussions throughout the project. Finally, we would like to thank the entire Machine Learning Research team at Apple for many helpful discussions and assistance with infra and data.

References

[1] Anonymous. V-JEPA: Latent video prediction for visual representation learning. In *Submitted to The Twelfth International Conference on Learning Representations*, 2023. 5

[2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. *arXiv preprint arXiv:2301.08243*, 2023. 2

[3] Yutong Bai, Xinyang Geng, Karttikeya Mangalam, Amir Bar, Alan Yuille, Trevor Darrell, Jitendra Malik, and Alexei A Efros. Sequential modeling enables scalable learning for large vision models. *arXiv preprint arXiv:2312.00785*, 2023. 2

[4] Peter Bandi, Oscar Geessink, Quirine Manson, Marcory Van Dijk, Maschenka Balkenhol, Meyke Hermsen, Babak Ehteshami Bejnordi, Byungjae Lee, Kyunghyun Paeng, Aoxiao Zhong, et al. From detection of individual metastases to classification of lymph node status at the patient level: the camelyon17 challenge. *IEEE Transactions on Medical Imaging*, 2018. 13

[5] Hangbo Bao, Li Dong, and Furu Wei. BEiT: Bert pre-training of image transformers. In *ICLR*, 2022. 2, 7, 8

[6] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. In *ICLR*, 2022. 2

[7] Miguel A Bautista, Artsiom Sanakoyeu, Ekaterina Tikhoncheva, and Bjorn Ommer. Cliquecnn: Deep unsupervised exemplar learning. *Advances in Neural Information Processing Systems*, 29, 2016. 2

[8] Sara Beery, Elijah Cole, and Arvi Gjoka. The iwildcam 2020 competition dataset. *arXiv preprint arXiv:2004.10340*, 2020. 13

[9] Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. A neural probabilistic language model. *Advances in neural information processing systems*, 13, 2000. 2

[10] Piotr Bojanowski and Armand Joulin. Unsupervised learning by predicting noise. In *International Conference on Machine Learning*, pages 517–526. PMLR, 2017. 2

[11] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – mining discriminative components with random forests. In *ECCV*, 2014. 13

[12] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale gan training for high fidelity natural image synthesis. *arXiv preprint arXiv:1809.11096*, 2018. 2

[13] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *preprint arXiv:2005.14165*, 2020. 1

[14] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. Deep clustering for unsupervised learning of visual features. In *ECCV*, 2018. 2

[15] Mathilde Caron, Piotr Bojanowski, Julien Mairal, and Armand Joulin. Unsupervised pre-training of image features on non-curated data. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2959–2968, 2019. 2

[16] Mathilde Caron, Ishan Misra, Julien Mairal, Priya Goyal, Piotr Bojanowski, and Armand Joulin. Unsupervised learning of visual features by contrasting cluster assignments. In *NeurIPS*, 2020. 4

[17] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *ICCV*, 2021. 2, 4, 8, 9

[18] Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. Generative pretraining from pixels. In *ICML*, 2020. 2, 8, 14

[19] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *ICML*, 2020. 2, 4, 7

[20] Xinlei Chen, Saining Xie, and Kaiming He. An empirical study of training self-supervised vision transformers. In *ICCV*, 2021. 3, 4

[21] Gordon Christie, Neil Fendley, James Wilson, and Ryan Mukherjee. Functional map of the world. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. 13

[22] M. Cimpoi, S. Maji, I. Kokkinos, S. Mohamed, and A. Vedaldi. Describing textures in the wild. In *CVPR*, 2014. 13

[23] Ekin D Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V Le. Autoaugment: Learning augmentation strategies from data. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 113–123, 2019. 14

[24] Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *ICML*. PMLR, 2023. 2, 4

[25] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 13

[26] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2018. 2, 7, 8

[27] Carl Doersch, Abhinav Gupta, and Alexei A Efros. Unsupervised visual representation learning by context prediction. In *ICCV*, 2015. 2

[28] Alexey Dosovitskiy, Jost Tobias Springenberg, Martin Riedmiller, and Thomas Brox. Discriminative unsupervised feature learning with convolutional neural networks. *Advances in neural information processing systems*, 27, 2014. 2, 4

[29] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. 2, 3, 7

[30] Alaaeldin El-Nouby, Gautier Izacard, Hugo Touvron, Ivan Laptev, Hervé Jegou, and Edouard Grave. Are large-scale datasets necessary for self-supervised pre-training? *arXiv preprint arXiv:2112.10740*, 2021. 9

[31] Jeffrey L Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990. 2

[32] Alex Fang, Albin Madappally Jose, Amit Jain, Ludwig Schmidt, Alexander Toshev, and Vaishaal Shankar. Data filtering networks. *arXiv preprint arXiv:2309.17425*, 2023. 1, 2, 3

[33] Samir Yitzhak Gadre, Gabriel Ilharco, Alex Fang, Jonathan Hayase, Georgios Smyrnis, Thao Nguyen, Ryan Marten, Mitchell Wortsman, Dhruba Ghosh, Jieyu Zhang, et al. Datacomp: In search of the next generation of multimodal datasets. *arXiv preprint arXiv:2304.14108*, 2023. 1, 2, 3

[34] Spyros Gidaris, Praveer Singh, and Nikos Komodakis. Unsupervised representation learning by predicting image rotations. *arXiv preprint arXiv:1803.07728*, 2018. 2

[35] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014. 2

[36] Priya Goyal, Dhruv Mahajan, Abhinav Gupta, and Ishan Misra. Scaling and benchmarking self-supervised visual representation learning. In *ICCV*, 2019. 2

[37] Priya Goyal, Quentin Duval, Isaac Seessel, Mathilde Caron, Mannat Singh, Ishan Misra, Levent Sagun, Armand Joulin, and Piotr Bojanowski. Vision models are more robust and fair when pretrained on uncurated images without supervision. *arXiv preprint arXiv:2202.08360*, 2022. 2

[38] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al. Bootstrap your own latent-a new approach to self-supervised learning. *NeurIPS*, 2020. 2, 4

[39] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *ICCV*, 2017. 3

[40] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *CVPR*, 2020. 2

[41] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *CVPR*, 2022. 2, 3, 4, 6, 7, 8, 9

[42] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification, 2017. 13

[43] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022. 2, 6

[44] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. 9

[45] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q Weinberger. Deep networks with stochastic depth. In *ECCV*, 2016. 2, 4

[46] iNaturalist 2018 competition dataset. iNaturalist 2018 competition dataset. https://github.com/visipedia/inat_comp/tree/master/2018, 2018. 13

[47] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *4th International IEEE Workshop on 3D Representation and Recognition (3dRR-13)*, Sydney, Australia, 2013. 13

[48] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 13

[49] Hugo Larochelle and Iain Murray. The neural autoregressive distribution estimator. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 29–37. JMLR Workshop and Conference Proceedings, 2011. 2

[50] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *ICML*, 2019. 5

[51] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *ICLR*, 2017. 14

[52] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017. 4, 14

[53] Tomas Mikolov, Martin Karafiát, Lukas Burget, Jan Cernockỳ, and Sanjeev Khudanpur. Recurrent neural network based language model. In *Interspeech*, 2010. 2

[54] Ishan Misra and Laurens van der Maaten. Self-supervised learning of pretext-invariant representations. In *CVPR*, 2020. 2

[55] Mehdi Noroozi and Paolo Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles. In *ECCV*, 2016. 2

[56] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016. 2

[57] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. In *NeurIPS*, 2018. 2

[58] Maxime Oquab, Timothée Darcet, Theo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Russell Howes, Po-Yao Huang, Hu Xu, Vasu Sharma, Shang-Wen Li, Wojciech Galuba, Mike Rabbat, Mido Assran, Nicolas Ballas, Gabriel Synnaeve, Ishan Misra, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2023. 2, 8, 9, 14

[59] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022. 1

[60] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar. Cats and dogs. In *CVPR*, 2012. 13

[61] Niki Parmar, Ashish Vaswani, Jakob Uszkoreit, Lukasz Kaiser, Noam Shazeer, Alexander Ku, and Dustin Tran. Image transformer. In *ICML*, 2018. 2

[62] Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell, and Alexei A Efros. Context encoders: Feature learning by inpainting. In *CVPR*, 2016. 2

[63] Xingchao Peng, Qinxun Bai, Xide Xia, Zijun Huang, Kate Saenko, and Bo Wang. Moment matching for multi-source domain adaptation. In *ICCV*, 2019. 13

[64] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 2019. 1, 2, 4

[65] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020. 4

[66] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 2020. 2

[67] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. In *International Conference on Machine Learning*, pages 8821–8831. PMLR, 2021. 6, 7

[68] Tim Salimans, Andrej Karpathy, Xi Chen, and Diederik P Kingma. Pixelcnn++: Improving the pixelcnn with discretized logistic mixture likelihood and other modifications. *arXiv preprint arXiv:1701.05517*, 2017. 2

[69] Claude E Shannon. Prediction and entropy of printed english. *Bell system technical journal*, 30(1):50–64, 1951. 2

[70] Mannat Singh, Quentin Duval, Kalyan Vasudev Alwala, Haoqi Fan, Vaibhav Aggarwal, Aaron Adcock, Armand Joulin, Piotr Dollár, Christoph Feichtenhofer, Ross Girshick, et al. The effectiveness of mae pre-pretraining for billion-scale pretraining. *arXiv preprint arXiv:2303.13496*, 2023. 2, 3, 8

[71] J. Taylor, B. Earnshaw, B. Mabey, M. Victors, and J. Yosinski. Rxrx1: An image set for cellular morphological variation across many experimental batches. In *ICLR*, 2019. 13

[72] Yonglong Tian, Olivier J Henaff, and Aäron van den Oord. Divide and contrast: Self-supervised learning from uncurated data. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10063–10074, 2021. 2

[73] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *ICML*, 2021. 2

[74] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. *arXiv preprint arXiv:2103.17239*, 2021. 2, 4, 5

[75] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. 1, 2, 3, 4, 7

[76] Aaron Van den Oord, Nal Kalchbrenner, Lasse Espeholt, Oriol Vinyals, Alex Graves, et al. Conditional image generation with pixelcnn decoders. *Advances in neural information processing systems*, 29, 2016. 2

[77] Aäron Van Den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *International conference on machine learning*, pages 1747–1756. PMLR, 2016. 2

[78] Aaron Van Den Oord, Oriol Vinyals, et al. Neurips. *Advances in neural information processing systems*, 2017. 6

[79] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017. 1, 2, 4

[80] Bastiaan S Veeling, Jasper Linmans, Jim Winkens, Taco Cohen, and Max Welling. Rotation equivariant cnns for digital pathology. In *Medical Image Computing and Computer Assisted Intervention–MICCAI 2018: 21st International Conference, Granada, Spain, September 16-20, 2018, Proceedings, Part II 11*, pages 210–218. Springer, 2018. 13

[81] Pascal Vincent, Hugo Larochelle, Isabelle Lajoie, Yoshua Bengio, Pierre-Antoine Manzagol, and Léon Bottou. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *Journal of machine learning research*, 11(12), 2010. 2

[82] Chen Wei, Karttikeya Mangalam, Po-Yao Huang, Yanghao Li, Haoqi Fan, Hu Xu, Huiyu Wang, Cihang Xie, Alan Yuille, and Christoph Feichtenhofer. Diffusion models as masked autoencoders. *arXiv preprint arXiv:2304.03283*, 2023. 2

[83] Xueting Yan, Ishan Misra, Abhinav Gupta, Deepti Ghadiyaram, and Dhruv Mahajan. ClusterFit: Improving Generalization of Visual Representations. In *CVPR*, 2020. 2

[84] Jiahui Yu, Zirui Wang, Vijay Vasudevan, Legg Yeung, Mojtaba Seyedhosseini, and Yonghui Wu. Coca: Contrastive captioners are image-text foundation models. *TMLR*, 2022. 5

[85] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow twins: Self-supervised learning via redundancy reduction. In *ICML*, 2021. 2

[86] Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *ICLR*, 2018. 14

[87] Richard Zhang, Phillip Isola, and Alexei A Efros. Colorful image colorization. In *ECCV*, 2016. 2

[88] Jinghao Zhou, Chen Wei, Huiyu Wang, Wei Shen, Cihang Xie, Alan Yuille, and Tao Kong. ibot: Image bert pre-training with online tokenizer. In *ICLR*, 2022. 2, 8, 9

A. Datasets

To assess the effectiveness and general applicability of the learned representations by AIM, we measure its recognition accuracy on a varied collection of 15 benchmarks in Table 6. The specifics of each benchmark can be found in Table 9. These benchmarks include datasets for tasks such as fine-grained recognition, medical imaging, satellite imagery, images in natural environments, and infographic images.

Dataset	train	test	classes
Imagenet-1k [25]	1,281,167	50,000	1000
iNAT-18 [46]	437,513	24,426	8142
CIFAR-10 [48]	50,000	10,000	10
CIFAR-100 [48]	50,000	10,000	100
Food101 [11]	75,750	25,250	101
DTD [22]	3,760	1,880	47
Pets [60]	3,680	3,669	37
Cars [47]	8,144	8,041	196
iWildCam [8]	129,809	14961	182
Camelyon17 [4]	302,436	34904	2
PCAM [80]	262,144	32768	2
RxRx1 [71]	40,612	9854	1139
EuroSAT [42]	16,200	5400	10
fMoW [21]	76,863	19915	62
Infograph [63]	36,023	15,582	345

Table 9. Evaluation benchmarks. We provide the references, the number of images in the train and test sets, and the number of categories of all the 15 recognition benchmarks used in this work.

B. Autoregression Patterns

We investigate different patterns that can be used to traverse an image during pre-training in Table 3b. All patterns used in this investigation are illustrated in Figure 8.

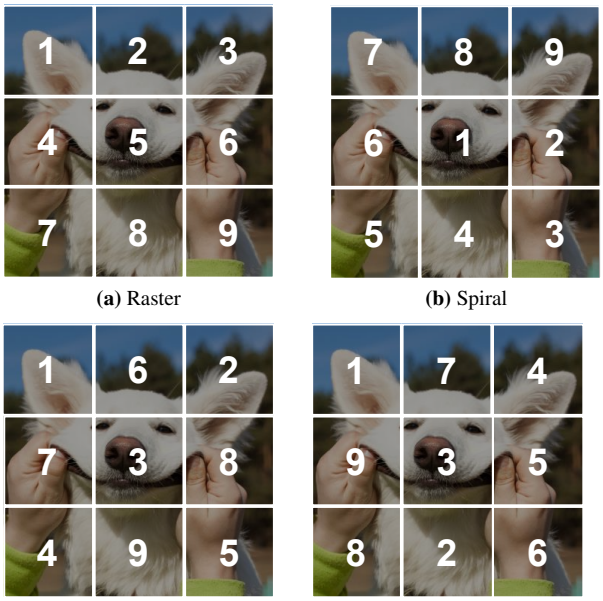
C. Additional Analysis

C.1. Raster pattern validation loss

In Figure 7, we noticed that the validation loss of the raster pattern across chunks surprisingly declined for the second chunk before increasing again. We investigated this further in Figure 9 and observed that this behavior is a side-effect of using the IN-1k validation set. In particular, we observed that the top rows of the image, aside from the first one, typically have a lower loss, whether the loss is computed over the regular image or its vertically flipped counterpart.

C.2. Downstream performance across layers

In Tables 7 and 6, we discussed the gain in downstream performance that can be achieved by probing shallower layers in the model rather than the last. We study this in more detail in Figure 10. We find that for all AIM variants, we extract the highest quality features, with respect to the downstream transfer, from layers roughly at two-thirds of the way into the model depth. However, it is important to note that the performance of deeper layers does not experience a steep decline and continues to exhibit strong performance.



(a) Raster **(b)** Spiral **(c)** Checkerboard **(d)** Random

Figure 8. Autoregression patterns. We illustrate the different autoregression patterns studied in this work including raster, spiral, checkerboard, and fixed random.

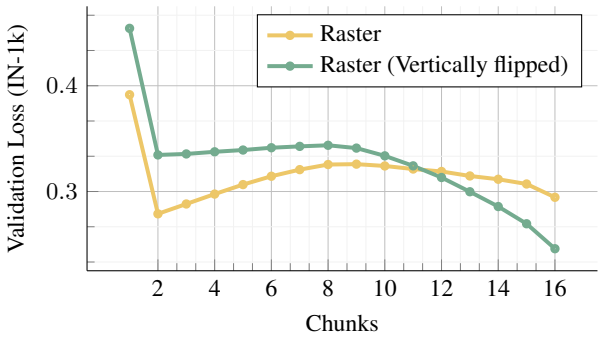


Figure 9. Raster pattern across patches. We compute the IN-1k validation loss per a chunk of 16 patches (*i.e.*, a row) for AIM-0.6B, pre-trained using a raster pattern. We measure the same loss for the vertically flipped images of the validation set. We observe that, for IN-1k validation set, the patches from the top rows in the image are easier to predict with lower loss, likely due to the concentration of background patches in that region.

D. Hyperparameters

Pre-training. AIM models of all capacities have been pre-trained using the same set of hyperparameters that are reported in Table 10. The AIM-0.6 model however has been

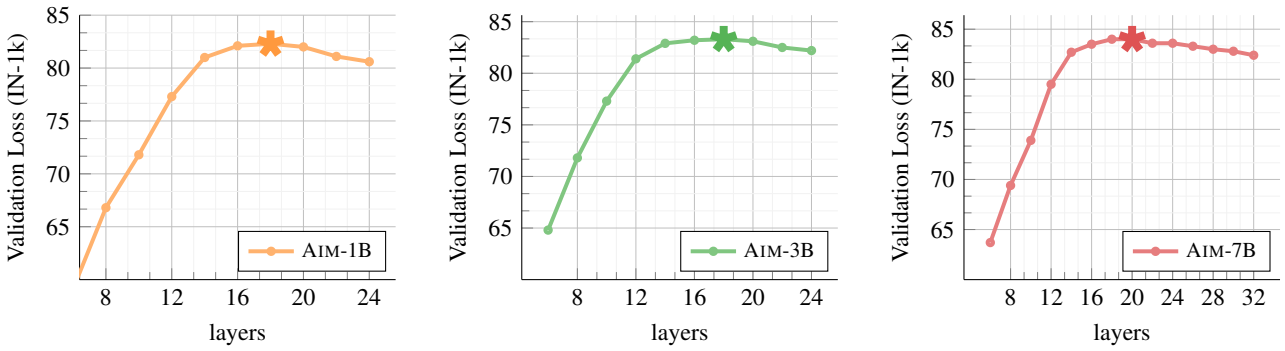


Figure 10. Downstream performance across layers. The highest quality features in terms of transfer to downstream recognition tasks can be extracted from layers different than the last, with the peak performance achieved by extracting from features roughly at two-thirds of the model depth. Deeper layers still retain a strong performance and no sharp decline is observed.

config	value
Optimizer	AdamW [52]
Optimizer Momentum	$\beta_1 = 0.9, \beta_2 = 0.95$
Peak learning rate	$1e^{-3}$
Minimum Learning rate	0.0
Weight decay	0.05
Batch size	4096
Patch size	(14, 14)
Gradient clipping	1.0
Warmup iterations	31,250
Total iterations	1,250,000
Learning rate schedule	cosine decay [51]
Augmentations:	
`RandomResizedCrop`	
`size`	224px
`scale`	[0.4, 1.0]
`ratio`	[0.75, 1.33]
`interpolation`	`Bicubic`
`RandomHorizontalFlip`	$p = 0.5$

Table 10. Pre-training hyperparameters All AIM variants of different capacities have been trained using the same set of hyperparameters detailed above.

config	IN-1k	Others
Optimizer	AdamW [52]	
Optimizer Momentum	$\beta_1 = 0.9, \beta_2 = 0.999$	
Peak learning rate grid	[1, 3, 5, 10, 15, 20, 40] $\times 1e^{-4}$	
Minimum Learning rate	$1e^{-5}$	
Weight decay	0.1	
Batch size	1024	512
Gradient clipping	3.0	
Warmup epochs	5	0
Epochs	50	100
Learning rate schedule	cosine decay [51]	
Augmentations:		
`RandomResizedCrop`		
`size`	224px	
`scale`	[0.08, 1.0]	
`ratio`	[0.75, 1.33]	
`interpolation`	`Bicubic`	
`RandomHorizontalFlip`	$p = 0.5$	
`Color Jitter`	0.3	
`AutoAugment` [23]	`rand-m9-mstd0.5-inc1`	

Table 11. Attentive probe hyperparameters. We detail the hyperparameters used for attentive probing AIM as well as the baselines. For all experiments, we search over different learning rate values and report the best for both AIM and baselines.

trained only for the shorter schedule of 500k iterations. We did not observe any instability while scaling the capacity of our model, thereby not requiring any further tuning of the optimization hyperparameters.

Attentive Probing. Downstream evaluation for AIM and the baselines has been primarily conducted via attentive probing as described in § 4. We report the hyperparameters used to probe all methods in Table 11. For a fair comparison with other baselines, we search over different values for the learning rate and report the best performance of each method similar to [58]. For AIM and other generative baselines, we average the features for the last 6 layers of the model before feeding them to the attention-probing head which leads to a modest gain in performance. Note that the descriptor dimensionality remains the same which is different from the practice of concatenating features similar to iGPT[18] which indirectly inflates the capacity of the evaluation head.

Low-rank adaptation. For LoRA finetuning, we use the same hyperparameters as reported in Table 11 in addition to mixup [86] (`alpha`=0.8). We apply LoRA adaptation, with `rank`=8, only to the parameters of the attention block. In particular, the weight matrices for the queries, values, and output projection.